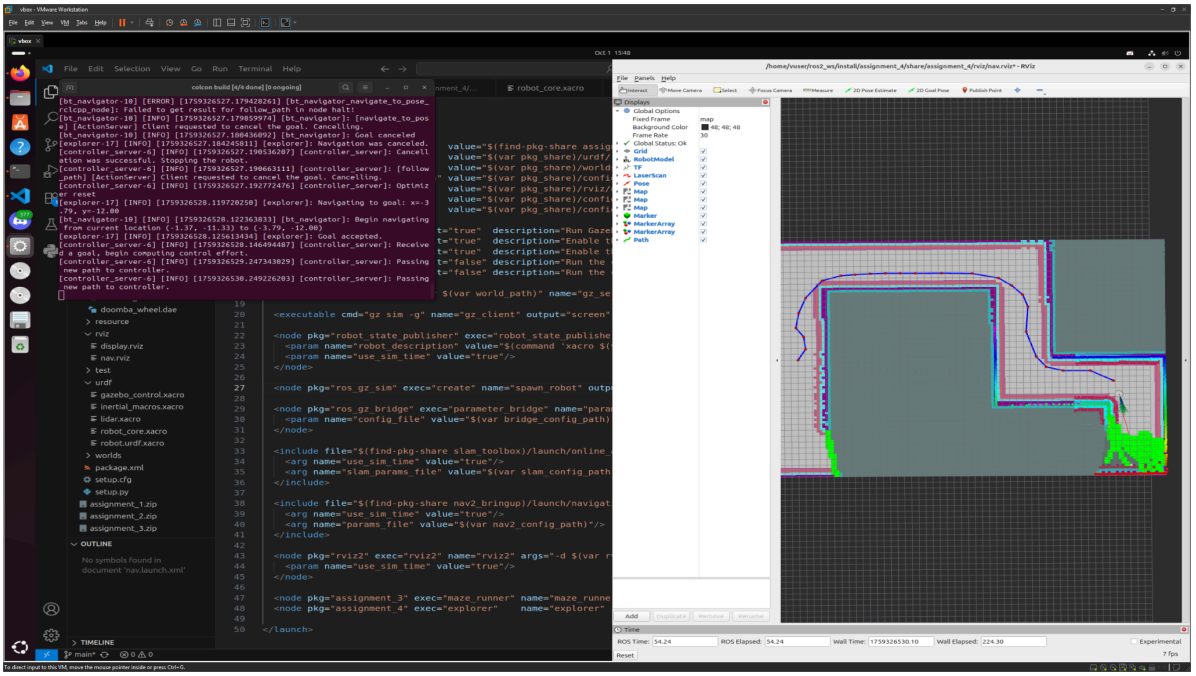
Assignment 4

This final assignment marks the culmination of your journey this semester, as you transition your robot from reactive control to full autonomy. Building upon the "Doomba" and the maze environment from Assignment 3, you will now implement the foundational capabilities of a truly autonomous mobile robot: mapping an unknown environment, navigating within that map, and exploring it to expand its knowledge. You will work with a single, evolving launch file and two new configuration files to manage the powerful slam_toolbox and Nav2 packages.



Prerequisites: Installing the Stacks

Before you begin, make sure you have installed the necessary ROS 2 packages by running the following commands in your terminal:

- sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup
- sudo apt install ros-jazzy-slam-toolbox

Task Breakdown

Before launching into the tasks, you should configure your LiDAR for optimal performance. For the best mapping results, update your lidar.xacro file to give the LiDAR a 360-degree field of view with 360 horizontal rays. This provides the SLAM and navigation systems with the rich data they need to operate effectively. You can experiment with different configurations if you want.

Task 1: Mapping the Maze with SLAM

Your first objective is to create an initial map of the maze by manually driving the robot. This map will serve as a baseline deliverable, confirming that your SLAM configuration is working correctly.

Getting Started — Nav2 1.0.0 documentation

(SLAM) Navigating While Mapping — Nav2 1.0.0 documentation

ROS2 SLAM Toolbox Tutorial Mobile Robot Simulation (SLAM Algorithm and Code)

- Create Configuration and Launch Files:
    - In your launch folder, create a new launch file named nav.launch.xml. This single file will be expanded upon in each task. For this first task, it should launch:
        - Your Gazebo simulation with maze.world.
        - The robot_state_publisher and the Gazebo bridge from your previous assignments.
        - The slam_toolbox node in its asynchronous mapping mode.
        - RViz for visualization.
    - **Note on Configuration**: You can launch the slam_toolbox node without a dedicated configuration file initially, but the performance will not be optimized for your robot. A better approach is to locate the default parameter file within the slam_toolbox package (/opt/ros/jazzy/share), copy it into your config folder as slam.yaml, and then modify it. This gives you a robust starting point. At a minimum, you must update key parameters, such as odom_frame, base_frame, and scan_topic, to match your robot's configuration.
- Build Your Map:
    - Launch your nav.launch.xml file. To create the map, you can either drive the robot manually using the teleop_twist_keyboard node or try running your mazerunner.py node from Assignment 3 to see it map by itself. While this wall-following isn't technically exploration, it's a great way to see the system coming together.
    - Whichever method you choose, carefully guide your robot through the entire maze. In RViz, if you've added map visualization, you should see the map being built in real-time. Aim for complete coverage of all corridors.
    - Once you are satisfied with the map's quality, save it. You can do this from the command line:
        - ros2 run nav2_map_server map_saver_cli -f maze_map
    - Alternatively, you can save it from within RViz by going to Panels > Add New

Panel > SlamToolboxPlugin. This will give you a GUI to save the map.

- This will generate maze_map.pgm and maze_map.yaml. Move these files into a new maps folder inside your package.
- **Important**: If you rename the map files, remember to also rename the image parameter inside the maze_map.yaml file, as it needs to link to the correct .pgm file. Also, remember to add the maps directory to your setup.py file so that your map files are included when the package is built.

Task 2: Autonomous Navigation with Nav2

Now, instead of driving manually, you will use Nav2's tools to send navigation goals to the robot. The robot will navigate to these goals while continuing to run SLAM and build the map. Your primary objective in this task is to configure and tune Nav2's costmap parameters for effective navigation.

Getting Started — Nav2 1.0.0 documentation

ROS2 Nav2 Robot Navigation Tutorial for Mobile Robot Simulation - YouTube

- Create Nav2 Configuration:
  - In your config folder, create a new file named nav2.yaml. This file will be used to tune the Nav2 stack, particularly the costmaps that allow the robot to plan paths around obstacles. A good starting point here is just as with the slam_toolbox, we can find the default nav2 parameter file that the nav2 node uses (/opt/ros/jazzy/share), and copy its content to our nav2.yaml file and then start configuring the values there.
- Expand the Launch File:
  - Modify your nav.launch.xml file to integrate the Nav2 stack. The standard way to do this is by including the navigation_launch.py file from the nav2_bringup package. This single action conveniently launches all the core Nav2 components (Planner, Controller, BT Navigator, etc.) needed for navigation.
  - Your launch file should now run Gazebo, your robot, slam_toolbox, and the included Nav2 launch file simultaneously. Remember to pass your nav2.yaml file as a parameter to the Nav2 launch include.
  - **Important**: Since slam_toolbox provides the map and the map-to-odom transform, you do not need to run map_server or amcl. Including navigation_launch.py (and not the larger bringup_launch.py) is the correct approach for this setup.
- Configure and Map with Nav2 Goals:

    - Relaunch your system with the updated nav.launch.xml.
    - In RViz, use the "2D Goal Pose" tool to set a target pose for the robot. Send the robot to various points within the maze.
    - Observe how the robot navigates and how the local and global costmaps are visualized in RViz. Your main goal here is to tune parameters in slam.yaml and nav2.yaml (e.g., inflation radius, obstacle layers) until the robot navigates through the maze corridors smoothly and safely, without touching the walls.

Task 3: Autonomous Frontier Exploration

In this final step, you will replace the manual goal-setting from Task 2 with a provided autonomous exploration node. This node will automatically identify frontiers on the map and send them as goals to Nav2, enabling the robot to autonomously map the entire maze.

- Integrate the Provided Explorer Node:
    - You will be provided with a Python script, explorer.py, which emulates the "frontier_lite" exploration algorithm. Place this file in the directory where executable scripts are located within your ROS 2 package.
    - While you are not writing the algorithm from scratch, you are expected to look through the explorer.py script and understand its logic. At a high level, the node subscribes to the /map topic, finds the closest frontier point, and uses a ROS 2 Action client to send that point as a goal to the Nav2 stack.
- Finalize the Launch File:
    - Add the explorer.py node to your nav.launch.xml file.
    - It's good practice to add a launch argument to easily switch between manual goal-setting (Task 2) and autonomous exploration (Task 3). For example:
        - <arg name="explore" default="false" description="Enable autonomous exploration"/>
    - Use a conditional group (<group if="$(var explore)">) to launch the explorer.py node only when this argument is set to true.
- Launch and Observe:
    - Run your final launch file in exploration mode: ros2 launch my_robot_description nav.launch.xml explore:=true.
    - Watch as your robot autonomously navigates and maps the entire maze without any human input. (IF you've configured the YAML files correctly)

Delivery

Your report should document the evolution of your configuration and launch files, explaining your logic for structuring the system.

- Configuration & Logic: Provide key snippets from your slam.yaml and nav2.yaml files, explaining the parameters you tuned and why. Based on your reading of the provided code, include a high-level explanation of how the explorer.py node interacts with slam_toolbox and Nav2 to achieve autonomous exploration.

- Visual Evidence:
    - A screenshot of your teleop-generated map from Task 1, along with the saved map files (.pgm and .yaml). These can be added to the report as an image and a text box.
    - A screenshot from Task 2, showing the robot navigating to a goal set in RViz, with the costmaps clearly visible (both global and local).
    - A short video demonstrating your robot autonomously exploring and mapping the maze in Task 3.

Good luck!